

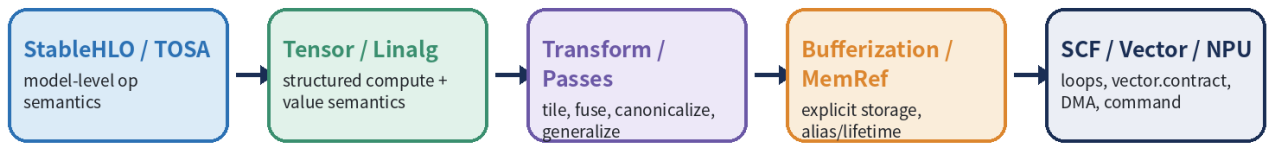
Lecture 07 - Linalg-on-Tensors

AI / NPU Compiler 를 위한 MLIR 기초 - 7 강 강의노트

핵심 실습/산출물: linalg.generic MatMul/Conv 표현, indexing_maps/iterator_types 분석표, NPU tiling/fusion 관점의 IR audit checklist.

Linalg-on-Tensors in an AI/NPU Compiler Pipeline

Keep structured tensor semantics long enough to tile, fuse, vectorize, and map to NPU kernels.



NPU compiler decisions enabled by Linalg

- indexing_maps expose data access functions for tile footprint analysis
- iterator_types distinguish parallel loops from reduction loops
- region body keeps scalar compute explicit for fusion and vectorization
- tensor outputs delay physical memory layout until bufferization

Figure 1. Linalg-on-Tensors 위치: high-level graph IR 과 bufferization/codegen 사이의 구조적 compute IR

1. 학습 목표

- Linalg-on-Tensors 가 StableHLO/TOSA 이후, Bufferization 이전에 위치하는 이유를 설명한다.
- linalg.generic 의 네 가지 핵심 필드: ins/outs, indexing_maps, iterator_types, region body 를 읽는다.
- MatMul 을 (m,n,k) loop domain 과 A(m,k), B(k,n), C(m,n) access map 으로 표현한다.
- Conv2D NHWC/HWCF 를 (n,oh,ow,f,kh,kw,c) loop domain 으로 모델링한다.
- NPU 관점에서 tile shape, SRAM tile footprint, accumulator pressure, epilogue fusion 후보를 도출한다.

2. 왜 Linalg-on-Tensors 인가?

AI model IR 은 보통 StableHLO/TOSA 처럼 operator semantic 이 강한 표현에서 시작한다. 그러나 NPU backend 는 단순히 op 이름만 보고 실행할 수 없다. 어떤 loop 가 parallel 인지, 어떤 loop 가 reduction 인지, 어떤 tensor index 를 읽고 쓰는지, producer/consumer fusion 이 가능한지를 알아야 한다. Linalg 는 이 정보를 structured op 로 보존한다.

관점	Linalg-on-Tensors 가 주는 정보	NPU compiler 활용
Iteration space	operand shape 와 indexing_maps 에서	tile size, output tile partition, halo 계

	loop domain 유도	산
Dependence	iterator_types 로 parallel/reduction 구분	PE mapping, accumulator allocation, reduction schedule
Compute payload	region body 로 scalar multiply/add/activation 표현	epilogue fusion, vectorization, custom ISA matching
Storage independence	tensor result 로 value semantic 유지	bufferization 전까지 layout/memory space 결정을 지연

중요한 설계 원칙은 “너무 빨리 loop/memref 로 낮추지 않는다”이다. Linalg 단계에서 tiling, fusion, generalization, vectorization 후보를 먼저 찾고, 이후 bufferization 에서 실제 memory space 와 layout 을 확정한다.

3. Linalg-on-Tensors 의 기본 모델

3.1 Tensor value semantic 과 Destination Style

Tensor type 은 값처럼 취급된다. linalg op 의 outs operand 는 결과 tensor 의 shape 와 destination contract 를 제공하고, op 는 tensor result 를 반환한다. 이 방식은 최적화 단계에서는 mutation 을 숨기고, bufferization 단계에서 redundant copy 를 제거하거나 in-place update 로 바꾸는 여지를 준다.

```
%init = tensor.empty() : tensor<8x32xf32>
%zero = linalg.fill ins(%c0 : f32)
               outs(%init : tensor<8x32xf32>) -> tensor<8x32xf32>
%C = linalg.matmul ins(%A, %B : tensor<8x16xf32>, tensor<16x32xf32>)
               outs(%zero : tensor<8x32xf32>) -> tensor<8x32xf32>
```

3.2 Named op 와 generic op

`linalg.matmul`, `linalg.conv\_2d\_nhwc\_hwcf` 같은 named op 는 의미를 간결하게 보존한다. 특정 변환에서는 named op 를 `linalg.generic` 으로 generalize 하여 indexing_maps 와 region body 를 명시적으로 드러낸다. NPU compiler 는 두 표현을 모두 다룰 수 있어야 한다.

표현	장점	주의점
Named op	패턴 매칭이 쉽고 matmul/conv semantic 이 명확함	특수 case 가 많아지면 backend pattern 이 분산될 수 있음
linalg.generic	access map, iterator, scalar payload 가 모두 노출됨	초심자에게 문법이 길고 shape/iterator 오류가 생기기 쉬움

4. linalg.generic Anatomy

Anatomy of linalg.generic

A structured operation = iteration space + access maps + iterator semantics + scalar payload region.

```
#matmul = {
  indexing_maps = [
    affine_map<(m,n,k) -> (m,k)>, // A
    affine_map<(m,n,k) -> (k,n)>, // B
    affine_map<(m,n,k) -> (m,n)>   // C
  ],
  iterator_types = ["parallel", "parallel", "reduction"]
}
%C = linalg.generic #matmul
  ins(%A, %B : tensor<MxKxf32>, tensor<KxNxf32>)
  outs(%init : tensor<MxNxf32>) {
^bb0(%a: f32, %b: f32, %c: f32):
  %p = arith.mulf %a, %b : f32
  %s = arith.addf %c, %p : f32
  linalg.yield %s : f32
} -> tensor<MxNxf32>
```

indexing_maps
Loop IVs -> tensor indices; key for tile footprint and layout analysis.

iterator_types
Parallel vs reduction dimensions; legal schedule and accumulator behavior.

ins / outs
Operands define iteration domain; outs carry destination/result shape.

region body
Scalar payload: elementwise math, reduction update, yield result.

Figure 2. linalg.generic 의 핵심 구성

필드	읽는 방법	NPU backend 질문
indexing_maps	loop IV tuple 을 각 operand index 로 변환	tile 이 읽는 A/B/filter/input footprint 는 얼마인가?
iterator_types	parallel/reduction loop semantics	어떤 축을 PE array 에 spatial mapping 하고 어떤 축을 accumulator 에 누적할까?
ins/outs	입력과 destination/result tensor	결과 tile shape 와 output memory layout 은 무엇인가?
region body	element-level scalar compute	mul-add, activation, requant, clamp 를 fusion 할 수 있는가?

`indexing\_maps` 는 NPU memory planner 의 핵심 입력이다. MatMul 에서 K tile 을 늘리면 A 와 B 의 SRAM footprint 가 증가하고 accumulator reuse 가 증가한다. Conv 에서는 OH/OW tile 이 input halo 를 만든다. 이 모든 정보는 affine map 에서 추론된다.

5. MatMul 을 linalg.generic 으로 표현하기

MatMul as linalg.generic: M x N parallel, K reduction

Loop domain (m,n,k) drives A(m,k), B(k,n), C(m,n).

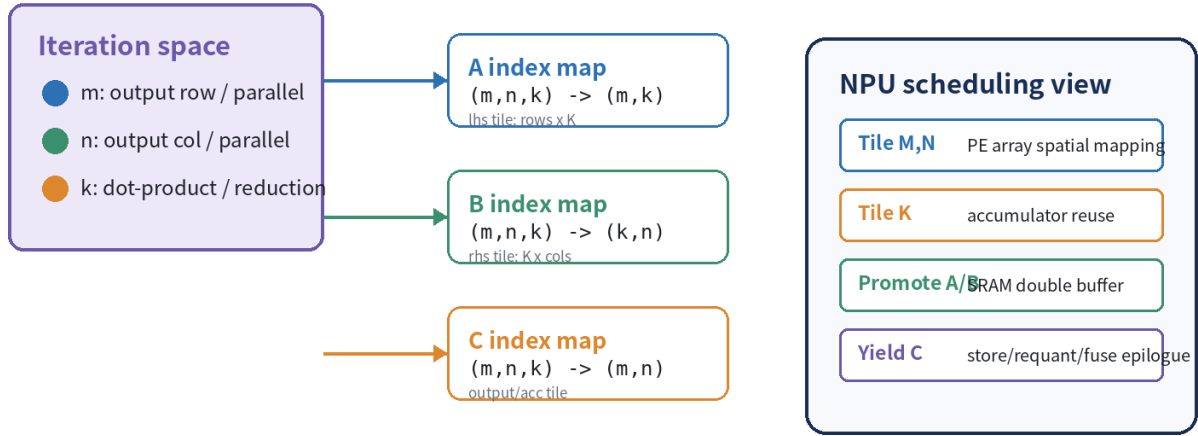


Figure 3. MatMul indexing maps 와 NPU scheduling 관점

MatMul $C[M,N] = \sum_k A[M,K] * B[K,N]$ 은 세 개의 loop dimension 을 갖는다. m 과 n 은 서로 독립적인 output element 를 만들기 때문에 parallel, k 는 같은 C(m,n)에 누적되므로 reduction 이다.

Loop dim	역할	Iterator type	NPU mapping hint
m	output row	parallel	PE rows 또는 output tile M
n	output column	parallel	PE columns 또는 output tile N
k	dot-product axis	reduction	accumulator reuse, K-slice streaming

```
#matmul_trait = {
    indexing_maps = [
        affine_map<(m, n, k) -> (m, k)>,
        affine_map<(m, n, k) -> (k, n)>,
        affine_map<(m, n, k) -> (m, n)>
    ],
    iterator_types = ["parallel", "parallel", "reduction"]
}

func.func @matmul_generic(%A: tensor<8x16xf32>,
                          %B: tensor<16x32xf32>) -> tensor<8x32xf32> {
    %c0 = arith.constant 0.0 : f32
    %init = tensor.empty() : tensor<8x32xf32>
    %zero = linalg.fill ins(%c0 : f32)
    outs(%init : tensor<8x32xf32>) -> tensor<8x32xf32>
    %C = linalg.generic #matmul_trait
        ins(%A, %B : tensor<8x16xf32>, tensor<16x32xf32>)
        outs(%zero : tensor<8x32xf32>) {
    ^bb0(%a: f32, %b: f32, %c: f32):
        %p = arith.mulf %a, %b : f32
        %s = arith.addf %c, %p : f32
    }
```

```

    linalg.yield %s : f32
} -> tensor<8x32xf32>
return %C : tensor<8x32xf32>
}

```

NPU 구현에서는 보통 M/N tile 을 PE array 의 spatial dimension 에 대응시키고 K tile 은 SRAM double-buffering 과 accumulator pressure 에 맞춰 분할한다. `linalg.yield` 는 output element 업데이트를 의미하므로 bias/add/relu/requant 같은 epilogue fusion 후보도 같은 region 또는 후속 consumer op 로 분석한다.

6. Conv2D NHWC/HWCF 를 linalg.generic 으로 표현하기

Conv2D NHWC/HWCF as linalg.generic

Loop domain (n,oh,ow,f,kh,kw,c): output axes are parallel; filter/channel axes are reductions.

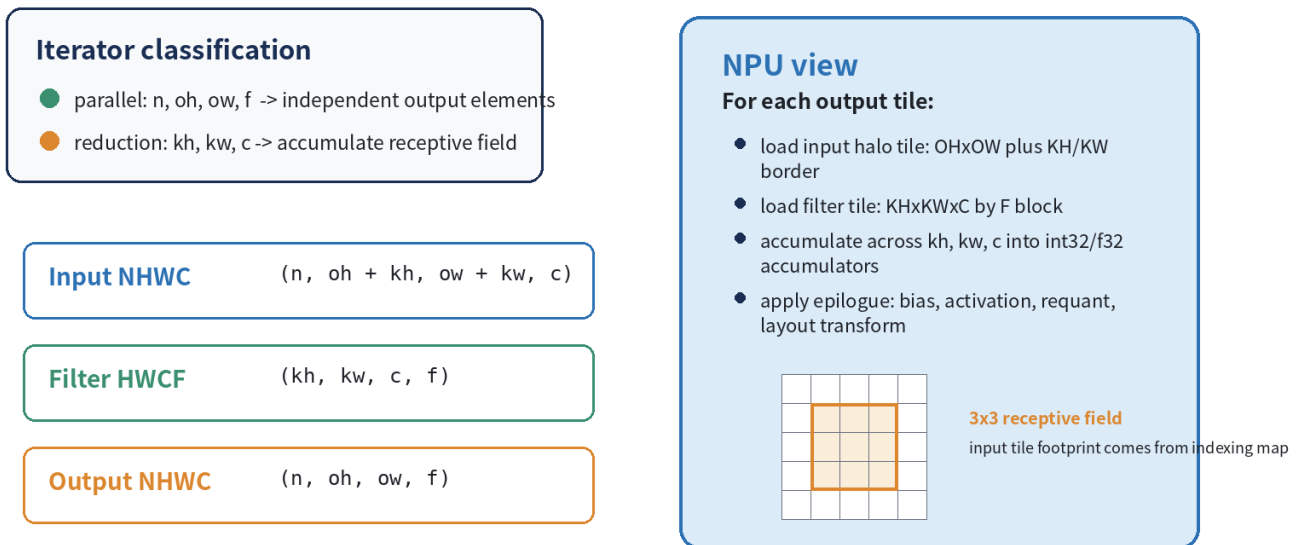


Figure 4. Conv2D NHWC/HWCF indexing maps

Conv2D output $O[n, oh, ow, f]$ 는 input $X[n, oh + kh, ow + kw, c]$ 와 filter $W[kh, kw, c, f]$ 의 reduction 이다. Stride/dilation/padding 이 들어오면 input indexing map 이 더 복잡해진다. 예를 들어 stride 가 2 이면 `oh \* 2 + kh` 형태가 된다.

Loop dim	의미	Iterator type	NPU 관점
n	batch	parallel	batch parallelism 또는 outer loop
oh/ow	output spatial	parallel	spatial tile, input halo 계산
f	output channel	parallel	filter/output channel tile, PE mapping
kh/kw	kernel spatial	reduction	filter tap loop, input reuse
c	input channel	reduction	MAC accumulation depth, activation/filter SRAM traffic

```

#conv_trait = {
    indexing_maps = [

```

```

    affine_map<(n, oh, ow, f, kh, kw, c) -> (n, oh + kh, ow + kw, c)>,
    affine_map<(n, oh, ow, f, kh, kw, c) -> (kh, kw, c, f)>,
    affine_map<(n, oh, ow, f, kh, kw, c) -> (n, oh, ow, f)>
  ],
  iterator_types = ["parallel", "parallel", "parallel", "parallel",
                    "reduction", "reduction", "reduction"]
}

func.func @conv2d_nhwc_hwc_f_generic(
  %input: tensor<1x30x30x16xf32>,
  %filter: tensor<3x3x16x32xf32>) -> tensor<1x28x28x32xf32> {
  %c0 = arith.constant 0.0 : f32
  %init = tensor.empty() : tensor<1x28x28x32xf32>
  %zero = linalg.fill ins(%c0 : f32)
                      outs(%init : tensor<1x28x28x32xf32>)
                      -> tensor<1x28x28x32xf32>
  %out = linalg.generic #conv_trait
    ins(%input, %filter : tensor<1x30x30x16xf32>, tensor<3x3x16x32xf32>)
    outs(%zero : tensor<1x28x28x32xf32>) {
    ^bb0(%x: f32, %w: f32, %acc: f32):
      %p = arith.mulf %x, %w : f32
      %s = arith.addf %acc, %p : f32
      linalg.yield %s : f32
    } -> tensor<1x28x28x32xf32>
  return %out : tensor<1x28x28x32xf32>
}

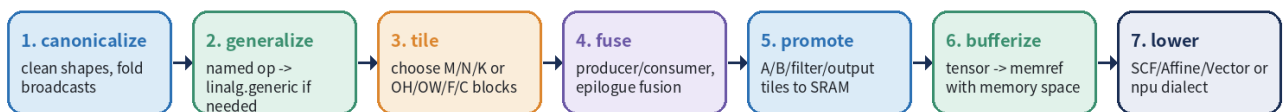
```

NPU compiler 는 Conv map 에서 input tile footprint 를 산출해야 한다. OHxOW output tile 이 있을 때 input 은 OH+KH-1, OW+KW-1 영역을 필요로 한다. 이 halo 때문에 단순 output tile 크기만으로 SRAM 필요량을 계산하면 오차가 생긴다.

7. Tiling/Fusion 전 Linalg Audit Checklist

From Linalg-on-Tensors to an NPU Kernel Candidate

Do high-level schedule decisions before tensor value semantics are erased by bufferization.



Schedule questions for 256+ TOPS-class Edge NPU

PE utilization	Does the tile shape expose enough parallel M/N/F work?
SRAM bandwidth	Does the input/filter tile fit with double buffering?
Accumulator pressure	How large is the reduction tile before spill?
Fusion legality	Can bias/activation/requant stay in the epilogue?
Layout	When do we convert NHWC/HWCF to hardware-preferred blocked layout?

Figure 5. Linalg-on-Tensors 에서 NPU kernel candidate 로 가는 설계 흐름

질문	확인할 IR 위치	실패 시 증상
모든 dynamic shape 가 backend 에서 지원되는가?	tensor type, tensor.dim, shape ops	runtime descriptor 가 과도하게 복잡해짐
parallel/reduction iterator 가 올바른가?	iterator_types	잘못된 parallelization 또는 atomic 필요
access map 이 hardware layout 과 충돌하지 않는가?	indexing_maps	transpose/copy 가 뒤늦게 삽입됨
epilogue fusion 이 가능한가?	producer/consumer linalg op chain	DRAM round-trip 증가
tile footprint 가 SRAM 에 맞는가?	indexing_maps + tile sizes	DMA burst 비효율 또는 spill
accumulator bit width 가 충분한가?	element type, reduction depth	overflow 또는 requant 정확도 저하

8. 실습

Exercise A - linalg.generic annotation

주어진 MatMul generic IR 에서 다음을 표시한다: loop domain, input maps, output map, parallel/reduction iterator, scalar payload, destination tensor.

Exercise B - MatMul named op 를 generic op 로 확장

`linalg.matmul` 을 직접 `linalg.generic` 으로 rewrite 한다. `%A`, `%B`, `%C` 의 shape 가 $M \times K$, $K \times N$, $M \times N$ 일 때 indexing_maps 를 작성하고 region body 를 완성한다.

Exercise C - Conv2D NHWC/HWCF map 작성

Input $1 \times 30 \times 30 \times 16$, Filter $3 \times 3 \times 16 \times 32$, Output $1 \times 28 \times 28 \times 32$ 에 대해 loop dimension 과 indexing_maps 를 작성한다. 이후 output tile $8 \times 8 \times 16$ 에 필요한 input/filter tile footprint 를 계산한다.

Exercise D - NPU kernel candidate 분석

MatMul+Bias+ReLU 또는 Conv+Bias+ReLU graph 를 보고 Linalg 단계에서 fusion 할 op, bufferization 이후로 미룰 op, NPU custom dialect 로 낫출 op 를 분류한다.

9. 7 강 핵심 Takeaways

- Linalg-on-Tensors 는 high-level semantic 과 low-level codegen 사이에서 구조적 loop/data mapping 을 보존한다.
- linalg.generic 은 indexing_maps, iterator_types, ins/outs, region body 네 부분으로 읽으면 된다.
- MatMul 은 (m,n,k) domain 과 parallel/parallel/reduction iterator 구성이 핵심이다.
- Conv NHWC/HWCF 는 output spatial/channel parallelism 과 kernel/input-channel reduction 을 분리해야 한다.
- NPU compiler 는 Linalg 단계에서 tile footprint, accumulator pressure, epilogue fusion, layout propagation 가능성을 판단해야 한다.

10. 참고자료

- MLIR Linalg Dialect: <https://mlir.llvm.org/docs/Dialects/Linalg/>

- MLIR Transform Tutorial Ch0 - Structured Linear Operations:
<https://mlir.llvm.org/docs/Tutorials/transform/Ch0/>
- MLIR Tensor Dialect: <https://mlir.llvm.org/docs/Dialects/TensorOps/>
- MLIR Passes: <https://mlir.llvm.org/docs/Passes/>